BrightLearn
AI & DATA SKILLS

STUDENT EXERCISE · UNION & UNION ALL

UNION & UNION ALL Exercise

Master combining result sets with UNION and UNION ALL. Ten questions, ten unique table pairs from a banking dataset — write the SQL, know the difference.

TOPIC	QUESTIONS
UNION & UNION ALL	10 + 3 bonus
LEVEL	**INCLUDES**
Beginner	20 tables · Expected output

Powered by CamScanner

00 Welcome & Quick Reference

Ten questions. Twenty tables. One goal: become fluent in the difference between UNION and UNION ALL. Every question shows you two tables from a banking dataset, gives you the task, and leaves you a box to write your SQL.

DEFINITION

The Exercise Domain

All twenty tables come from a single fictional **African retail bank called NexBank**. You'll combine account holders, transactions, staff, product lists, complaints, and more — alternating between UNION and UNION ALL so the difference becomes automatic.

THE TWO OPERATORS — KNOW THESE COLD

UNION

- Combines results from both SELECT statements
- Removes duplicate rows automatically
- Slightly slower — extra deduplication step
- Use for: unique lists, master tables
- Think: DISTINCT result set

UNION ALL

- Combines results from both SELECT statements
- Keeps ALL rows — including duplicates
- Faster — skips the deduplication step
- Use for: audit logs, ledgers, full history
- Think: append rows, no filtering

★ THE THREE RULES — BREAK ANY OF THESE AND YOUR QUERY FAILS

- **Both SELECT statements must return the same number of columns** — you cannot UNION a 2-column query with a 3-column query.
- **Matching columns must have compatible data types** — you cannot combine a text column with a number column in the same position.
- **Column names come from the first SELECT** — alias your columns in the first query if you want specific output names.

SYNTAX REFERENCE

Powered by CamScanner

```
-- UNION: removes duplicates
SELECT column1, column2  FROM table_A
UNION
SELECT column1, column2  FROM table_B;

-- UNION ALL: keeps every row
SELECT column1, column2  FROM table_A
UNION ALL
SELECT column1, column2  FROM table_B;
```

★ HOW TO WORK THROUGH THIS EXERCISE

Read both tables carefully. Read the task. Decide which operator the task requires. Write your SQL in the answer box. Run it in your SQL environment (PostgreSQL, SQLite, MySQL, SQL Server, or BigQuery all work). Compare your output to the expected columns. Repeat until all ten are done — then answer the three bonus questions at the end.

Powered by CamScanner

01 Questions 01 – 05 (UNION)

The next five questions all use UNION. Each pair of tables contains some overlapping rows. Your job is to return one unique combined list — no duplicates allowed.

Table: branch_sandton_accounts

account_id	account_holder
A001	Nomvula Dlamini
A002	David Mokoena
A003	Lerato Sithole
A004	Peter Nkosi

Table: branch_rosebank_accounts

account_id	account_holder
A003	Lerato Sithole
A004	Peter Nkosi
A005	Zanele Khumalo
A006	Thabo Motha

QUESTION 01 · USE UNION

Combine the account holders from both branches into one unique list. Each person should appear only once, even if they hold accounts at both branches.

EXPECTED COLUMNS

`account_id, account_holder, city`

WRITE YOUR SQL HERE

```
SELECT account_id, account_holder, 'Sandton' AS City
FROM branch_sandton_accounts
UNION
SELECT account_id, account_holder, 'Rosebank' AS City
FROM branch_rosebank_accounts;
```

★ CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Table: savings_products **Table: current_products**

Powered by CamScanner

product_code	product_name
SV01	Basic Savings
SV02	Premium Savings
SV03	Youth Savings
SV04	Business Savings

product_code	product_name
CR01	Standard Current
CR02	Gold Current
SV03	Youth Savings
CR03	Business Current

QUESTION 02 · USE UNION

NexBank offers two product lists — one for savings accounts and one for current accounts. Build a unique product catalogue that shows every product the bank offers, without repeating any product that appears in both lists.

EXPECTED COLUMNS

product_code, product_name, product_type

WRITE YOUR SQL HERE

```
SELECT product_code, product_name, 'Savings' AS
    product_type
FROM Savings_product
UNION
SELECT product_code, product_name, 'Current' AS
    product_type
FROM Current_products;
```

★ CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Table: retail_banking_staff

staff_id	staff_name
S001	Mpho Radebe
S002	Brian Tshabalala
S003	Aisha Patel
S004	Kabelo Moabelo

Table: corporate_banking_staff

staff_id	staff_name
S003	Aisha Patel
S005	Nandi Dube
S006	Sipho Khumalo
S004	Kabelo Moabelo

Powered by CamScanner

QUESTION 03 · USE UNION

The HR team needs a unique list of all staff who work in either Retail Banking or Corporate Banking. Some staff are assigned to both. Each staff member should appear only once.

EXPECTED COLUMNS

`staff_id, staff_name, email`

WRITE YOUR SQL HERE

```
SELECT staff_id, staff_name, CAST(NULL AS STRING)
        AS email
FROM retail_banking_staff
UNION
SELECT staff_id, staff_name, CAST(NULL AS STRING)
        AS email
FROM corporate_banking_staff
ORDER BY staff_id;
```

★ CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Table: mobile_branch_cities

city_code	city_name
C01	Johannesburg
C02	Pretoria
C03	Cape Town
C04	Durban

Table: digital_branch_cities

city_code	city_name
C03	Cape Town
C05	Polokwane
C06	Port Elizabeth
C01	Johannesburg

QUESTION 04 · USE UNION

Marketing needs a unique list of all cities where NexBank serves customers — either through its mobile branches or its digital branch (which serves customers online regardless of city). No city should appear twice.

EXPECTED COLUMNS

`city_code, city_name, region`

Powered by CamScanner

WRITE YOUR SQL HERE

```
SELECT city_code, city_name, 'Mobile Branch' AS region
UNION FROM mobile_branch_cities
UNION
SELECT city_code, city_name, 'Digital Branch' AS region
FROM digital_branch_cities;
```

★ CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Table: push_notification_targets

customer_id	customer_name
C1001	Nomsa Zwane
C1002	Andile Buthelezi
C1003	Fatima Mahomed
C1004	Ryno van Zyl

Table: inapp_banner_targets

customer_id	customer_name
C1003	Fatima Mahomed
C1005	Thandeka Cele
C1006	Samuel Nkosi
C1002	Andile Buthelezi

QUESTION 05 · USE UNION

The digital marketing team ran two campaigns last month — one via push notification and one via in-app banner. Build a unique list of every customer who was targeted. If a customer received both, they must appear only once.

EXPECTED COLUMNS

`customer_id, customer_name, segment`

WRITE YOUR SQL HERE

```
SELECT customer_id, customer_name, 'Marketing' AS
    Segment
FROM push_notification_targets
UNION
SELECT customer_id, customer_name, 'Marketing'
AS segment
FROM inapp_banner_targets
```


Powered by CamScanner

★ CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Powered by CamScanner

02 Questions 06 – 10 (UNION ALL)

The next five questions all use UNION ALL. Every row from both tables must appear in the output — including duplicates. Notice how the result sets are larger than what UNION would return.

Table: atm01_transactions

txn_id	account_id	amo
T1001	A001	500.00
T1002	A002	1200.00
T1003	A001	300.00
T1004	A003	750.00

Table: atm02_transactions

nttxn_id	account_id	amo
T1003	A001	300.00
T1005	A004	900.00
T1006	A002	450.00
T1007	A005	150.00

QUESTION 06 · USE UNION ALL

The fraud team needs the complete log of all ATM transactions from machine ATM-01 and machine ATM-02. Every transaction must appear — even if the same transaction appears in both logs due to a system sync error.

EXPECTED COLUMNS

`transaction_id, account_id, amount, transaction_date`

WRITE YOUR SQL HERE

```
SELECT txn_id AS transaction_id, account_id, amount,
       CURRENT_DATE AS transaction_date
FROM atm01_transactions
UNION ALL
SELECT txn_id AS transaction_id, account_id, amount,
       CURRENT_DATE AS transaction_date
FROM atm02_transactions;
```

* CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Table: **Table:**

Powered by CamScanner

gauteng_loan_applications

app_id	customer_id	H
LA001	C1001	Vehicle Loan
LA002	C1002	Personal
LA003	C1003	Home Loan
LA004	C1004	

western_cape_loan_applications

app_id	customer_id	
LA003	C1003	Personal
LA005	C1005	Vehicle Loan
LA006	C1006	Business
LA007	C1007	Home Loan

QUESTION 07 · USE UNION ALL

The credit team wants a full list of all loan applications received from the Gauteng region and the Western Cape region. Every application must be included — duplicates are allowed because both regions may have captured the same application.

EXPECTED COLUMNS

`application_id, customer_id, loan_type, amount_requested`

WRITE YOUR SQL HERE

```
SELECT app_id AS application_id, customer_id, loan_type,
       NULL AS amount_requested
FROM gauteng_loan_applications
UNION ALL
SELECT app_id AS application_id, customer_id,
       loan_type, NULL AS amount_requested
FROM western_cape_loan_applications;
```

★ CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Table: email_complaints

complaint_id	customer_id
EC001	C2001
EC002	C2002
EC003	C2003
EC004	C2004

Table: app_complaints

complaint_id	customer_id
AC001	C2005
AC002	C2001
AC003	C2006
AC004	C2007

Powered by CamScanner

QUESTION 08 · USE UNION ALL

The customer experience team needs a full record of every complaint logged this month, whether it came via email or via the mobile app. Every complaint must appear — the team needs all of them to calculate total complaint volumes.

EXPECTED COLUMNS

`complaint_id, customer_id, category, logged_date`

WRITE YOUR SQL HERE

```
SELECT complaint_id, 'Email' AS customer_id, 'Email' AS
    category, CURRENT_DATE AS logged date
FROM email_complaints
UNION ALL
SELECT complaints_id, customer_id, 'APP' AS category
    CURRENT_DATE AS logged date
FROM app_complaint.
```

★ CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Table: april_payments

payment_id	account_id	amount
PAY001	A001	12500.00
PAY002	A002	4800.00
PAY003	A003	9200.00
PAY004	A004	3300.00

Table: may_payments

payment_id	account_id	amount
PAY005	A001	12500.00
PAY006	A005	7600.00
PAY007	A002	5100.00
PAY008	A006	2800.00

QUESTION 09 · USE UNION ALL

Finance needs to reconcile all payment records from April and May into one table for the half-year report. Every payment from both months must appear — the total row count matters for reconciliation.

EXPECTED COLUMNS

`payment_id, account_id, amount, payment_date`

Powered by CamScanner

WRITE YOUR SQL HERE

```
SELECT payment_id, account_id, amount, '2025-04-01'
    AS payment_date
FROM april_payments
UNION ALL
SELECT payment_id, account_id, amount, '2025-05-01'
    AS payment_date
FROM may_payments;
```

★ CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Table: debit_entries

entry_id	account_id	entry_type
DR001	A001	Debit
DR002	A002	Debit
DR003	A003	Debit
DR004	A004	Debit

Table: credit_entries

entry_id	account_id	entry_type
CR001	A001	Credit
CR002	A005	Credit
CR003	A002	Credit
CR004	A006	Credit

QUESTION 10 · USE UNION ALL

The accounting system records debits and credits in two separate tables. Combine all entries into one general ledger for the audit. Every single entry from both tables must appear — no rows should be dropped.

EXPECTED COLUMNS

`entry_id, account_id, entry_type, amount, entry_date`

WRITE YOUR SQL HERE

```
SELECT entry_id, account_id, entry_type, NULL AS amount,
    CURRENT_DATE AS entry_date
FROM debit_entries
SELECT entry_id, account_id, entry_type, NULL
    AS amount, CURRENTDATE AS AS
    entry_date
FROM credit_entries;
```


Powered by CamScanner

★ CHECK — RUN YOUR QUERY AND VERIFY AGAINST THE EXPECTED COLUMNS ABOVE

Powered by CamScanner

03 Bonus Questions

These three questions are written — no SQL needed. They test your understanding of why you would choose one operator over the other, and what goes wrong when you break the rules.

BONUS 01

A fraud analyst wants a unique list of customers who appeared in either the January or February high-risk watch lists. Should they use UNION or UNION ALL? Explain why.

YOUR ANSWER

use UNION because unique customer list is required and duplicates must be removed

BONUS 02

The auditing team wants to count every individual transaction that passed through the bank in Q1 — even if the same transaction was recorded in two systems on the same day. Should they use UNION or UNION ALL? Explain why.

YOUR ANSWER

use UNION ALL because every transaction must be counted, including duplicates.

Powered by CamScanner

BONUS 03

A developer writes this query: SELECT account_id, account_holder FROM branch_sandton_accounts UNION SELECT account_id, account_holder, city FROM branch_rosebank_accounts; The query fails. What is wrong, and how would you fix it?

YOUR ANSWER

The query fails because the SELECT statements return different numbers of columns (2 vs 3). Make both SELECT statements return the same number of columns with compatible data types

Powered by CamScanner

04 Self-Check & Submission

Before you submit your work, run through this checklist. The highest marks always go to students who verify their own results before handing anything in.

★ YOUR SUBMISSION CHECKLIST

- **All 10 queries run without errors** in your SQL environment.
- **UNION queries return no duplicate rows** — row count equals the count of DISTINCT rows across both tables.
- **UNION ALL queries include all rows** — row count equals the sum of both tables' row counts.
- **Output columns match the spec** — same names, same order as the Expected Columns on each card.
- **Bonus answers are in full sentences** — they should explain the why, not just name the operator.

★ WHAT TO SUBMIT

- **One .sql file** with all 10 queries labelled -- Q1 through -- Q10.
- **Screenshots or text output** of each query result.
- **Bonus answers** in the boxes above or in a separate .txt file.
- **Your SQL environment** noted at the top of the file (e.g. PostgreSQL 16 / SQLite / BigQuery).

★ THE ONE THING TO REMEMBER

UNION and UNION ALL do the same thing — they combine rows from two queries. The only difference is what they do with duplicates. UNION removes them. UNION ALL keeps them. Every time you write one of these operators, ask yourself: do I want only unique records, or do I want every single row? Answer that question first, then pick the operator. It will never be wrong again.

Powered by CamScanner